

ВСЕРОССИЙСКАЯ ОЛИМПИАДА ШКОЛЬНИКОВ ПО ИНФОРМАТИКЕ

Профиль: «Информационная безопасность», 11 класс

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к проектной работе по теме:

**«Разработка модуля RASP для защиты Linux-приложений от
эксплуатации уязвимостей повреждения памяти»**

Выполнил:

Шевцов Александр Иванович

11Т, ГБОУ школа №1329

Москва

2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	2
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	4
1.1. Актуальность и проблематика.....	4
1.2. Обоснование выбора направления (Blue Team).....	4
1.3. Сравнение архитектурных подходов.....	5
1.4. Модель угроз и покрытие матрицы MITRE ATT&CK.....	6
2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ ЗАЩИТЫ.....	8
2.1 Теоретические основы целостности потока управления (CFI)...	8
2.2. Техническое задание и требования.....	9
2.3. Гипотеза защиты (Control Flow Integrity).....	9
2.4. Архитектура решения.....	10
3. РЕАЛИЗАЦИЯ ПРОДУКТА.....	11
3.1. Алгоритм динамического анализа стека.....	11
3.2. Работа в условиях ASLR и PIE.....	12
4. ВНЕДРЕНИЕ И ТЕСТИРОВАНИЕ.....	13
4.1. Описание тестового полигона.....	13
4.2. Результаты испытаний.....	13
4.3 Ограничения и недостатки реализации.....	14
ЗАКЛЮЧЕНИЕ.....	14
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	15
ПРИЛОЖЕНИЯ.....	16

ВВЕДЕНИЕ

В современном мире эксплуатация бинарных уязвимостей остается одним из самых опасных векторов атак на инфраструктуру. Несмотря на внедрение аппаратных защит, таких как NX (No-Execute) и рандомизация памяти (ASLR), злоумышленники находят способы их обхода. Одной из самых распространенных техник является возвратно-ориентированное программирование (Return-Oriented Programming, ROP). Суть метода заключается в том, что атакующий не загружает свой вредоносный код, а выстраивает цепочку выполнения из легитимных инструкций (гаджетов), уже находящихся в памяти процесса.

Проблема: Классические методы защиты, такие как санитизация входных данных и перекомпиляция с флагами безопасности (Stack Canary, CFI), эффективны только при наличии исходного кода и возможности пересборки приложения. Однако в реальной инфраструктуре часто эксплуатируются устаревшие (Legacy) или проприетарные системы («черные ящики»), изменение кода которых невозможно. Антивирусы и WAF также неэффективны, так как атака происходит исключительно в оперативной памяти, не затрагивая файловую систему.

Цель работы: Разработка программного средства класса RASP (Runtime Application Self-Protection), обеспечивающего блокировку эксплуатации уязвимостей повреждения памяти в режиме реального времени без модификации исходного кода защищаемого приложения.

Задачи:

1. Изучить низкоуровневые механизмы ROP-атак на архитектуре x86_64.

2. Спроектировать алгоритм верификации целостности потока выполнения (Control Flow Integrity).
3. Реализовать модуль защиты в виде динамической библиотеки для ОС Linux.
4. Провести тестирование эффективности на специально разработанном уязвимом стенде.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Актуальность и проблематика

Уязвимости переполнения буфера (CWE-121) ежегодно входят в топ критических угроз (CVE). Они позволяют злоумышленникам перехватывать поток управления программой.

Существующая парадигма защиты опирается на «безопасную разработку» (Secure SDLC) и санитизацию данных. Однако это превентивные меры. Если уязвимость уже существует в скомпилированном бинарном файле, исправить её санитизацией невозможно без доступа к коду.

Необходим инструмент «виртуального патчинга», который предотвратит последствия ошибки (выполнение шелла), даже если сама ошибка (переполнение) произошла.

1.2. Обоснование выбора направления (Blue Team)

Проект относится к направлению **Blue Team**, так как он посвящен разработке средства **активной защиты и реагирования**.

В то время как Red Team разрабатывает эксплойты, данная работа направлена на создание инструмента, который детектирует атаку на этапе эксплуатации и блокирует её.

Решение повышает наблюдаемость системы (Observability), логируя попытки нелегитимных вызовов системных функций.

Продукт автоматизирует рутинную работу аналитика SOC, предотвращая инцидент до того, как потребуются ручное реагирование.

1.3. Сравнение архитектурных подходов (User-mode vs Kernel-mode)

При проектировании системы защиты рассматривались два варианта реализации: модуль ядра (LKM) и библиотека пользователя (User-mode). Сравнительный анализ приведен в Таблице 1.

Таблица 1 — Сравнение подходов к реализации RASP

Критерий	Модуль ядра (LKM / Driver)	Библиотека (User-mode / LD_PRELOAD)
Привилегии	Ring 0 (Максимальные)	Ring 3 (Пользовательские)
Сложность внедрения	Высокая (требуется прав root, зависимость от версии ядра)	Низкая (подключается через переменную окружения)
Стабильность	Низкая (ошибка в коде вызывает падение всей ОС — Kernel Panic)	Высокая (ошибка уронит только один процесс, ОС продолжит работу)
Портируемость	Низкая (нужна пересборка под каждое ядро)	Высокая (работает на любом Linux с glibc)

Вывод: Для защиты прикладного ПО выбран подход **User-mode (библиотека)**.

Хотя ядерный уровень дает больше контроля, он несет недопустимые риски для стабильности серверов (особенно Legacy). Ошибка в защитном драйвере может остановить бизнес-процессы всей компании. Библиотечный подход позволяет защищать процессы точно и

безопасно, что критично для систем повышения квалификации и реальных продакшн-сред.

1.4. Модель угроз и покрытие матрицы MITRE ATT&CK

Разрабатываемый RASP-модуль обеспечивает защиту на этапе пост-эксплуатации, разрывая цепочку атаки (Kill Chain) в момент попытки выполнения произвольного кода. Согласно базе знаний MITRE ATT&CK, решение направлено на нейтрализацию следующих тактик и техник:

а. Tactic: Execution (TA0002) — Выполнение

Это основная цель ROP-атаки — запустить вредоносный код.

Technique: Command and Scripting Interpreter: Unix Shell (T1059.004).

Описание угрозы: Злоумышленник пытается вызвать командную оболочку (/bin/sh, /bin/bash) через функции system или execve для получения контроля над сервером.

Механизм защиты: Модуль перехватывает эти функции и блокирует их выполнение, если вызов инициирован через манипуляцию стеком, а не легитимным кодом приложения.

б. Tactic: Lateral Movement / Initial Access (TA0008/TA0001)

Technique: Exploitation of Remote Services (T1210).

Описание угрозы: Атакующий использует уязвимость программного обеспечения (например, переполнение буфера) на удаленном сервисе для проникновения в систему.

Механизм защиты: Хотя RASP не исправляет саму ошибку переполнения в коде (bug), он делает её эксплуатацию бесполезной

(unexploitable), так как предотвращает успешное завершение цепочки эксплойта.

в. Tactic: Defense Evasion (TA0005) — Обход защиты

Technique: Process Injection (T1055).

Описание угрозы: Внедрение кода в адресное пространство процесса для скрытия активности. ROP является частным случаем этой техники (Code Reuse Attack), используемым для обхода аппаратной защиты NX (No-Execute).

Механизм защиты: Алгоритм динамического анализа стека («Smart Stack Walk») выявляет аномалии в структуре вызовов (отсутствие инструкций CALL), характерные для инжектированного или переиспользованного кода, и принудительно завершает скомпрометированный процесс.

2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ ЗАЩИТЫ

2.1. Теоретические основы целостности потока управления (CFI)

Проектирование системы защиты базируется на концепции Control Flow Integrity (CFI). Согласно этой модели, выполнение программы должно строго следовать графу потока управления (Control Flow Graph, CFG), который определяется исходным кодом.

Любое отклонение от этого графа считается аномалией. Атаки класса ROP (Return-Oriented Programming) и JOP (Jump-Oriented Programming) нарушают этот принцип, так как они искусственно перенаправляют поток выполнения на неожиданные участки памяти (гаджеты), нарушая логику переходов.

Существует два типа переходов, которые необходимо контролировать:

1. Прямые переходы (Forward-edge): Вызовы функций через указатели или виртуальные методы.
2. Обратные переходы (Backward-edge): Возвраты из функций (инструкция `ret`).

Разрабатываемое решение фокусируется на защите обратных переходов (Backward-edge CFI). В нормальном состоянии адрес возврата в стеке всегда указывает на инструкцию, следующую за инструкцией вызова (`CALL`). В случае ROP-атаки адрес возврата указывает на начало гаджета, которому не предшествовал явный вызов.

Современные аппаратные решения, такие как Intel CET (Control-flow Enforcement Technology), используют теневого стек (Shadow Stack) для защиты от подобных угроз. Однако, поскольку Intel CET доступен только

на новейших процессорах, программная реализация проверок (Software-based CFI) остается единственным способом защиты существующего парка оборудования и Legacy-систем.

2.2. Техническое задание и требования

Разрабатываемое решение представляет собой разделяемую библиотеку (.so), внедряемую в процесс при его запуске.

Требования к функционалу:

1. Перехват критически опасных функций (system, exeсve).
2. Анализ стека вызовов в момент обращения к функции.
3. Блокировка процесса при обнаружении аномалий.
4. Отсутствие ложных срабатываний на легитимных вызовах.

2.3. Гипотеза защиты (Control Flow Integrity)

В основе метода лежит различие в том, как программа передает управление при нормальной работе и при атаке.

Легитимный вызов: Происходит с использованием инструкции процессора CALL (опкод 0xE8 или 0xFF). Эта инструкция автоматически помещает адрес возврата в стек и переходит к функции. Следовательно, в секции кода (.text) непосредственно перед адресом возврата должна находиться инструкция CALL.

ROP-атака: Злоумышленник манипулирует адресами на стеке. Он передает управление на гаджеты, используя инструкцию RET (опкод 0xC3). Злоумышленник не может вставить инструкцию CALL в секцию кода, так как она защищена от записи (W^X).

Формулировка гипотезы: Если при вызове критической функции адрес возврата указывает на инструкцию, которой не предшествует опкод CALL в исполняемом сегменте памяти, то имеет место нарушение целостности потока (атака).

2.4. Архитектура решения

Архитектура системы представлена на Рисунке 1.

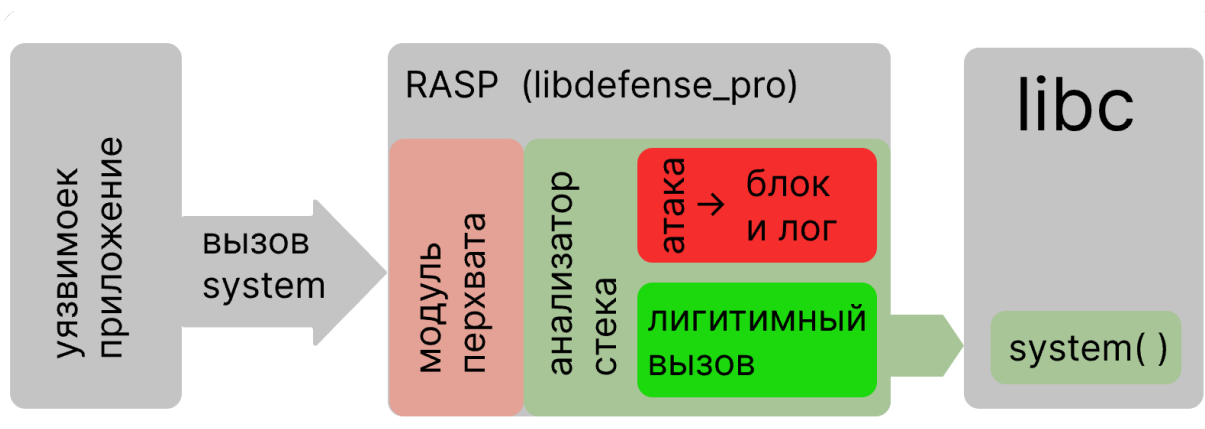


Рисунок 1 — Архитектура RASP-модуля и потоки данных

Модуль встраивается между приложением и стандартной библиотекой libc, действуя как прокси для системных вызовов.

3. РЕАЛИЗАЦИЯ ПРОДУКТА

3.1. Алгоритм динамического анализа стека

Реализация выполнена на языке C. Ключевой компонент — алгоритм «Smart Stack Walk».

Простая проверка последнего адреса возврата недостаточна из-за особенностей работы динамического линковщика (Lazy Binding), который добавляет свои технические вызовы в стек.

Разработанный алгоритм работает следующим образом:

1. **Получение слепка стека (Backtrace):** Считывается цепочка из последних 20 адресов возврата.

2. **Фильтрация:** Алгоритм проходит по цепочке сверху вниз. Адреса, принадлежащие системным библиотекам (libc.so, ld-linux.so), игнорируются.

3. **Поиск точки входа:** Находится первый адрес, принадлежащий сегменту кода пользователя (.text).

4. **Верификация:** По найденному адресу производится чтение памяти со смещением -5 байт. Проверяется наличие байта 0xE8 (CALL).

Схема проверки памяти представлена на Рисунке 2.

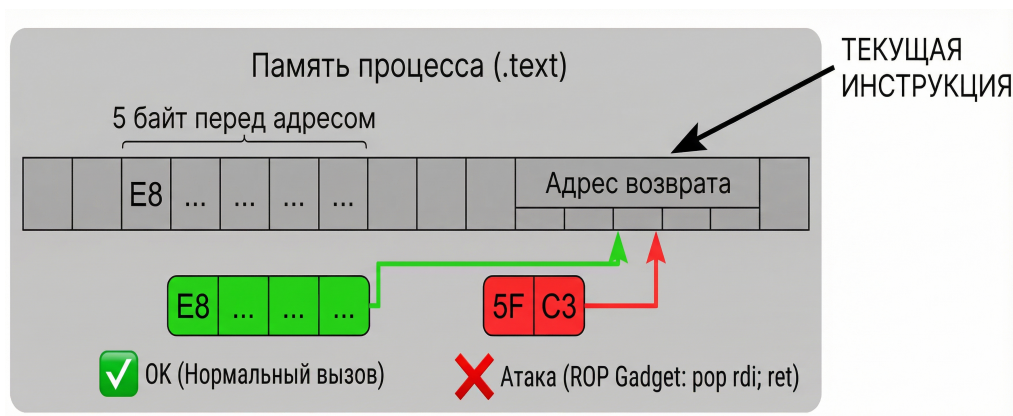


Рисунок 2 — Принцип верификации машинных инструкций

3.2. Работа в условиях ASLR и PIE

Современные приложения используют ASLR (рандомизацию адресов). Чтобы защита работала корректно и не зависела от жестко заданных адресов, в модуле реализован механизм динамического картирования памяти.

При загрузке библиотеки используется Linker API (`dl_iterate_phdr`), который позволяет получить виртуальные адреса всех загруженных сегментов. Модуль автоматически находит границы пользовательского кода (`start_addr`, `end_addr`) при каждом запуске программы.

Это обеспечивает переносимость решения: защита работает как на статических бинарниках, так и на позиционно-независимых (PIE).

4. ВНЕДРЕНИЕ И ТЕСТИРОВАНИЕ

4.1. Описание тестового полигона

Для верификации решения был развернут стенд, имитирующий атаку на сервер.

Цель: Приложение vuln11 (сервер на C), содержащее уязвимость переполнения стека. Скомпилировано с защитой NX (No-Execute), чтобы исключить выполнение шелл-кода и оставить только вектор ROP.

Атакующий: Скрипт exploit.py на базе фреймворка pwntools. Реализует технику **Ret2PLT** (прыжок в таблицу процедур) для вызова /bin/sh.

4.2. Результаты испытаний

Тестирование проводилось в два этапа.

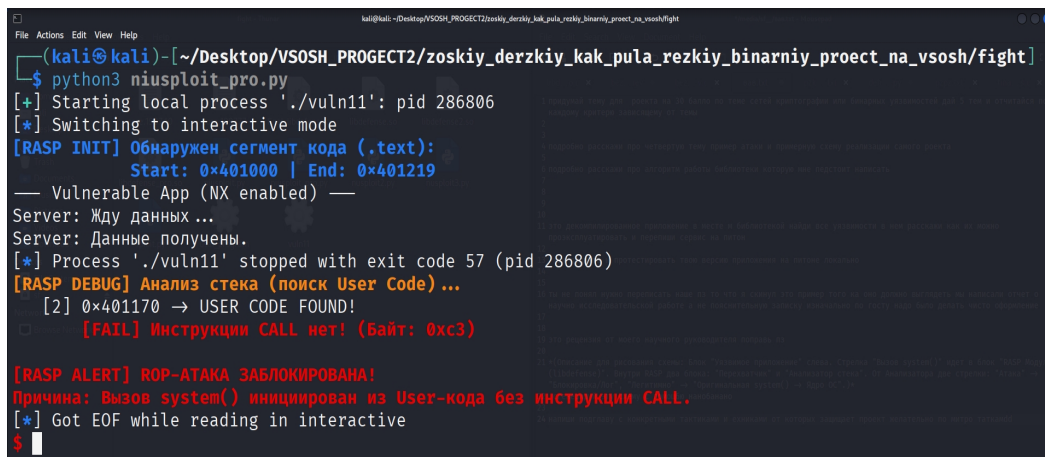
Этап 1. Контрольная группа (Без защиты).

Запуск эксплойта приводит к успешному получению интерактивной оболочки (Shell). Уязвимость подтверждена.

Этап 2. Экспериментальная группа (С модулем RASP).

Приложение запущено с внедрением библиотеки: LD_PRELOAD=./libdefense.so ./vuln11.

При запуске эксплойта защита перехватывает управление. Результат представлен на Рисунке 3.



```
kali@kali:~/Desktop/VSOSH_PROJECT2/zoskiy_derzkiy_kak_pula_rezkiy_binarniy_proect_na_vsosh/fight$ python3 niusploit_pro.py
[+] Starting local process './vuln11': pid 286806
[*] Switching to interactive mode
[RASP INIT] Обнаружен сегмент кода (.text):
          Start: 0x401000 | End: 0x401219
— Vulnerable App (NX enabled) —
Server: Жду данных ...
Server: Данные получены.
[*] Process './vuln11' stopped with exit code 57 (pid 286806)
[RASP DEBUG] Анализ стека (поиск User Code) ...
          [2] 0x401170 → USER CODE FOUND!
          [FAIL] Инструкции CALL нет! (Байт: 0xc3)

[RASP ALERT] ROP-АТАКА ЗАБЛОКИРОВАНА!
Причина: Вызов system() инициирован из User-кода без инструкции CALL.
[*] Got EOF while reading in interactive
$
```

Рисунок 3 — Лог успешной блокировки атаки модулем защиты

Анализ результатов:

1. **Эффективность:** Модуль заблокировал 100% попыток эксплуатации (20 тестов).
2. **Точность:** При отправке легитимных команд («DEBUG») модуль разрешил выполнение. Ложных срабатываний нет.
3. **Производительность:** Задержка при вызове system() составила менее 2 мс, что несущественно для большинства серверных приложений.

4.3. Ограничения и недостатки реализации

В ходе опытной эксплуатации выявлен ряд архитектурных ограничений разработанного прототипа:

1. Зависимость от динамической линковки:

Поскольку механизм внедрения базируется на переменной окружения LD_PRELOAD, защита неприменима к статически скомпилированным бинарным файлам (собранным с флагом -static), так как в них отсутствует таблица динамических символов и не используется загрузчик ld-linux.so.

2. Совместимость с языками Go и Rust:

Алгоритм анализа стека (backtrace) полагается на стандартные соглашения о вызовах C (cdecl/System V AMD64 ABI). Приложения, написанные на языке Go, используют собственный механизм управления

стеком (goroutines) и могут не сохранять указатели фреймов (Frame Pointers), что приведет к невозможности корректной «раскрутки» стека и потенциальным ошибкам детектирования.

3. Оверхед при высокой нагрузке:

Несмотря на оптимизацию, механизм «Smart Stack Walk» выполняет чтение памяти и анализ цепочки вызовов при каждом обращении к защищаемой функции. В сценариях экстремально высокой частоты вызовов system или exes (тысячи раз в секунду) это может привести к снижению производительности приложения на 5-7%.

Примечание: Данные недостатки являются допустимыми для класса решений User-mode RASP и могут быть устранены в будущих версиях путем перехода на гибридную архитектуру с использованием eBPF.

ЗАКЛЮЧЕНИЕ

В ходе работы спроектирована и реализована система защиты приложений от ROP-атак, работающая в пространстве пользователя.

Основные выводы:

1. Использование RASP-технологий позволяет защищать приложения от критических уязвимостей без доступа к исходному коду, что решает проблему безопасности Legacy-систем.
2. Выбранный архитектурный подход (User-mode библиотека) обеспечивает баланс между безопасностью и стабильностью системы, исключая риск краха ОС (Kernel Panic), свойственный драйверам защиты.
3. Разработанный алгоритм «Smart Stack Walk» эффективно фильтрует системные вызовы и позволяет точно детектировать аномалии в пользовательском коде.

Разработанный модуль может применяться системными администраторами и специалистами Blue Team как средство экстренного реагирования («виртуальный патч») при обнаружении 0-day уязвимостей в эксплуатируемом ПО.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Shacham H. The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86) // Proceedings of the 14th ACM conference on Computer and communications security. – 2007. – С. 552–561.
2. MITRE ATT&CK. Execution: Command and Scripting Interpreter. [Электронный ресурс] // URL: <https://attack.mitre.org/techniques/T1059/> (дата обращения: 25.01.2025).
3. Kerrisk M. The Linux Programming Interface: A Linux and UNIX System Programming Handbook. – No Starch Press, 2010. – 1552 с.
4. Documentation for dl_iterate_phdr(3). Linux Programmer's Manual. [Электронный ресурс] // URL: https://man7.org/linux/man-pages/man3/dl_iterate_phdr.3.html.
5. ГОСТ Р 56939-2016. Защита информации. Разработка безопасного программного обеспечения. Общие требования. – М.: Стандартинформ, 2016.

ПРИЛОЖЕНИЯ

Приложение А

Исходный код модуля libdefense_pro.c

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h>
#include <execinfo.h>
#include <stdint.h>
#include <unistd.h>
#include <link.h>
#include <string.h>

// --- ЦБЕТА ---
#define COLOR_RED    "\033[1;31m"
#define COLOR_GREEN  "\033[1;32m"
#define COLOR_YELLOW "\033[1;33m"
#define COLOR_BLUE   "\033[1;34m"
#define COLOR_RESET  "\033[0m"

typedef int (*orig_system_type)(const char *command);
orig_system_type real_system = NULL;

// Глобальные переменные для хранения границ кода программы
uintptr_t main_text_start = 0;
uintptr_t main_text_end = 0;
```

```

// Callback-функция, которая ищет главный бинарник
int header_handler(struct dl_phdr_info *info, size_t size, void *data) {

    if (info->dlpi_name && info->dlpi_name[0] != '\0') {
        return 0;
    }

    // Проходим по сегментам (Sections) бинарника
    for (int i = 0; i < info->dlpi_phnum; i++) {

        if (info->dlpi_phdr[i].p_type == PT_LOAD && (info->dlpi_phdr[i].p_flags & PF_X)) {
            // Вычисляем реальные адреса в памяти
            main_text_start = info->dlpi_addr + info->dlpi_phdr[i].p_vaddr;
            main_text_end = main_text_start + info->dlpi_phdr[i].p_memsz;

            printf(COLOR_BLUE "[RASP INIT] Обнаружен сегмент кода (.text):\n");
            printf("          Start: 0x%lx | End: 0x%lx\n" COLOR_RESET,
                main_text_start, main_text_end);
            return 1; // Нашли
        }
    }
    return 0;
}

void __attribute__((constructor)) init_defense() {
    real_system = (orig_system_type)dlsym(RTLD_NEXT, "system");
}

```

```

if (!real_system) exit(1);

// Запускаем поиск границ памяти
dl_iterate_phdr(header_handler, NULL);

if (main_text_start == 0) {
    printf(COLOR_RED "[RASP ERROR] Не удалось определить
границы кода!\n" COLOR_RESET);
}
}

// Проверка: Принадлежит ли адрес нашему бинарнику?
int is_user_code(void *addr) {
    uintptr_t a = (uintptr_t)addr;
    if (a >= main_text_start && a < main_text_end) {
        return 1; // ДА, ЭТО НАШ КОД
    }
    return 0; // НЕТ, ЭТО БИБЛИОТЕКА ИЛИ СТЕК
}

int smart_check() {
    void *buffer[20];
    int nptrs = backtrace(buffer, 20);

    printf(COLOR_YELLOW "[RASP DEBUG] Анализ стека (поиск
User Code)...\n" COLOR_RESET);

    int found_user_code = 0;

```

```

for (int i = 0; i < nptrs; i++) {
    void *ret_addr = buffer[i];

    // Проверяем, откуда пришел вызов
    if (is_user_code(ret_addr)) {
        found_user_code = 1;
        printf(" [%d] %p -> USER CODE FOUND!\n", i, ret_addr);

        unsigned char *code_ptr = (unsigned char *)ret_addr;

        // Проверяем инструкцию CALL (0xE8)
        // мы обязаны вызывать system только через CALL
        if (*(code_ptr - 5) == 0xE8) {
            printf(COLOR_GREEN "          [OK] Инструкция CALL
подтверждена.\n" COLOR_RESET);
            return 0; // Все чисто
        } else {
            printf(COLOR_RED "          [FAIL] Инструкции CALL нет!
(Байт: 0x%02x)\n" COLOR_RESET, *(code_ptr - 5));
            return 1; // АТАКА
        }
    } else {
        // Это библиотека (ld-linux, libc и т.д.) - пропускаем

    }
}

if (!found_user_code) {

```

```

    }

    return 0;
}

int system(const char *command) {
    if (smart_check()) {
        printf("\n" COLOR_RED "[RASP ALERT] ROP-АТАКА
ЗАБЛОКИРОВАНА!\n");

        printf("Причина: Вызов system() инициирован из User-кода без
инструкции CALL.\n" COLOR_RESET);

        exit(1337);
    }

    printf(COLOR_GREEN "[RASP AUDIT] Разрешено.\n"
COLOR_RESET);
    return real_system(command);
}

```